

Southeast University Examination Paper (A)

Course Name Principles of Compiling Examination Term 13-14-2 Score _____
Related Major Computer Science & Technology Examination Form Close test Test Duration 150 Mins

There are 8 problems in this paper. You can write the answers in English or Chinese on the attached paper sheets.

1. Please construct context-free grammars **with ϵ -free productions** for the following language (10%).

$\{a^m \omega b^m \mid m \geq 0 \text{ and } \omega \in (c,d,e,f)^* \text{ and the numbers of d's and e's and f's occurred in } \omega \text{ are even}\}$

2. Please construct a **DFA with minimum states** for the following regular expression. (15%)

$((a|b)^*(ab)(a|b)^*)^*(ab)(a|b)^*(a|b)$

3. Please **eliminate the left recursions (if there are)** and **extract maximum common left factors (if there are)** from the following context free grammar, and then decide **the resulted grammar** is whether a LL(1) grammar by **constructing the related LL(1) parsing table.**(15%)

$S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid \text{while } E \text{ do } S \mid \text{id} = F$

$E \rightarrow E \text{ and } E | E \text{ or } E | \text{not } E | (E) | b$

$F \rightarrow F + F | F * F | (F) | n$

4. Please **construct a LR(1) parsing table for the following ambiguous grammar with your own defined additional conditions (You determine the required additional conditions by yourself).**(15%)

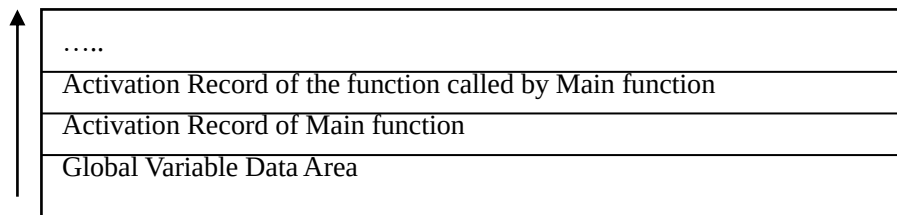
$E \rightarrow E \theta_1 E | E \theta_2 E | E \theta_3 E | (E) | id$

5. Please construct **an annotated parse tree** for the input string $(2+3*4+@5)+@6*7$ where the syntax-directed definition is as following (10%):

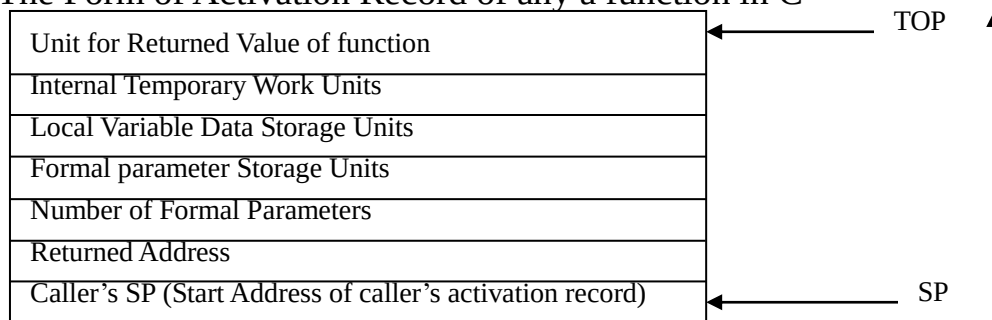
Productions	Semantic Rules
$E \rightarrow E_1 * F$	$\{E.val = E_1.val * F.val\}$
$E \rightarrow F$	$\{E.val = F.val\}$
$F \rightarrow F_1 + G$	$\{F.val = F_1.val + G.val\}$
$F \rightarrow G$	$\{F.val = G.val\}$
$G \rightarrow (E)$	$\{G.val = E.val\}$
$G \rightarrow i$	$\{G.val = i.lexval\}$
$G \rightarrow @i$	$\{G.val = 0 - i.lexval\}$

6. We assume that the storage organization and the form of activation record used in C language program run-time stack storage allocation are as following. Please **construct the run-time stack map when it gets the maximum size at the second time** for the following C program (10%).

Storage Organization of C Language



The Form of Activation Record of any a function in C



```
#include <stdio.h>
```

```
int x,y;
```

```
int main()
```

```
{
```

```
    x=10;
```

```
    y=f(x);
```

```
}
```

```
int f(int m)
```

```
{
```

```
    if (m>=0)
```

```
        if (m==0) return(1)
```

```

    else if (m==1) return(1)
    else if (m==2) return(2)
    else {
        int t1,t2,t3,t;
        t1=f(m-1);
        t2=f(m-2);
        t3=f(m-3);
        t=t1+t2+t3;
        return(t)
    }
else return(-1)
}

```

Notes: 1) Here we assume that the caller's sp of Main function is the start address of global variable data area, and the returned address in the activation record of a function (including Main function) is filled by the operating system automatically, you might not care it.

2) The initial value of variable X is 10, the start address of stack used in the program is K.

3) The stack map may get its maximum size for several times, here we ask you draw the stack map at maximum size for the second time.

7. Please translate the following program fragment into **Quadruple sequence using short circuit code and back-patching techniques.** (15%)

```

i=1;
while (i<=10) {
    j=1;
    while (j<=10) {
        k=1;
        m=0;
        while (k<=10) {
            if (A[i,k]==0 || B[k,j]==0) k=k+1;
            else {
                m=m+A[i,k]*B[k,j];
            }
        }
    }
    i=i+1;
}

```

```

        k=k+1
    }
    C[i,j]=m;
    j=j+1;
}
i=i+1
}

```

Notes: 1) Here we assume that the declaration of array A, array B and array C are array [1..10,1..10], each data element of **array A, array B and array C would use 4 storage unit**, and the start address of array A's storage area is addrA, the start address of array B's storage area is addrB, the start address of array C's storage area is addrC.

2) The related semantic rules are described as followings, where NXINSTR means "No. of Next Instruction", the No. of first instruction is (1).

$E \rightarrow i \quad \{E \cdot TC = NXINSTR; E \cdot FC = NXINSTR + 1;$

$GEN('if' i.place '<>0' 'goto 0'); GEN('goto 0')\}$

$E \rightarrow E_a \text{ rop } E_b \quad \{E \cdot TC = NXINSTR; E \cdot FC = NXINSTR + 1;$

$GEN('if' E_a.place \text{rop.op } E_b.place 'goto 0'); GEN('goto 0')\}$

$E \rightarrow (E^{(1)}) \quad \{E \cdot TC = E^{(1)} \cdot TC; E \cdot FC = E^{(1)} \cdot FC\}$

$E \rightarrow \text{not } E^{(1)} \quad \{E \cdot TC = E^{(1)} \cdot FC; E \cdot FC = E^{(1)} \cdot TC\}$

$E^A \rightarrow E^{(1)} \text{ and } \quad \{BACKPATCH(E^{(1)} \cdot TC, NXINSTR); E^A \cdot FC = E^{(1)} \cdot FC;\}$

$E \rightarrow E^A E^{(2)} \quad \{E \cdot TC = E^{(2)} \cdot TC; E \cdot FC = \text{MERG}(E^A \cdot FC, E^{(2)} \cdot FC)\}$

$E^0 \rightarrow E^{(1)} \text{ or } \quad \{BACKPATCH(E^{(1)} \cdot FC, NXINSTR); E^0 \cdot TC = E^{(1)} \cdot TC;\}$

$E \rightarrow E^0 E^{(2)} \quad \{E \cdot FC = E^{(2)} \cdot FC; E \cdot TC = \text{MERG}(E^0 \cdot TC, E^{(2)} \cdot TC)\}$

$C \rightarrow \text{if } E \text{ then } \{ \text{BACKPATCH}(E \cdot \text{TC}, \text{NXINSTR}); C \cdot \text{CHAIN} = E \cdot \text{FC}; \}$

$T \rightarrow C S^{(1)} \text{ else } \{ q = \text{NXINSTR}; \text{GEN}(\text{'goto } 0\text{'}) ;$

$\text{BACKPATCH}(C \cdot \text{CHAIN}, \text{NXINSTR});$

$T \cdot \text{CHAIN} = \text{MERG}(S^{(1)} \cdot \text{CHAIN}, q) \}$

$S \rightarrow T S^{(2)} \quad \{ S \cdot \text{CHAIN} = \text{MERG}(T \cdot \text{CHAIN}, S^{(2)} \cdot \text{CHAIN}) \}$

$S \rightarrow C S^{(1)} \quad \{ S \cdot \text{CHAIN} = \text{MERG}(C \cdot \text{CHAIN}, S^{(1)} \cdot \text{CHAIN}) \}$

$W \rightarrow \text{while} \quad \{ W \cdot \text{LABEL} = \text{NXINSTR} \}$

$W^d \rightarrow W E \text{ do } \{ \text{BACKPATCH}(E \cdot \text{TC}, \text{NXINSTR}); W^d \cdot \text{CHAIN} = E \cdot \text{FC};$

$W^d \cdot \text{LABEL} = W \cdot \text{LABEL}; \}$

$S \rightarrow W^d S^{(1)} \quad \{ \text{BACKPATCH}(S^{(1)} \cdot \text{CHAIN}, W^d \cdot \text{LABEL});$

$\text{GEN}(\text{'goto' } W^d \cdot \text{LABEL}); S \cdot \text{CHAIN} = W^d \cdot \text{CHAIN} \}$

$A \rightarrow S; \quad \{ \text{BACKPATCH}(S \cdot \text{CHAIN}, \text{NXINSTR}) \}$

$S \rightarrow A S^{(1)} \quad \{ S \cdot \text{CHAIN} = S^{(1)} \cdot \text{CHAIN} \}$

(NXINSTR means “No. of Next Instruction”)

8. Please **construct the DAG** for the following basic block, optimize the block and **rewrite the block** in optimized code form. Note that we assume **only Variable L would be used later**(10%)

B=3

D=A+C

E=A*C

F=D+E

G=B*F

H=A+C

$$I=A*C$$

$$J=H+I$$

$$K=B*5$$

$$L=K+J$$

$$M=L$$